

The Time Dependent Traveling Salesman Planning Problem in Controlled Airspace

Fabio Furini¹, Carlo Alfredo Persiani², Paolo Toth³

¹LAMSADE, Université Paris-Dauphine, Place du Maréchal de Lattre de Tassigny, 75775 Paris, France

`fabio.furini@dauphine.fr`

²ENAV S.p.A, Italian Agency for Air Navigation Services, Via Salaria 716 Roma, Italy

`carlo.persiani@enav.it`

³DEI, University of Bologna, Viale Risorgimento 2, 40136 Bologna, Italy

`paolo.toth@unibo.it`

Author's version. This is the author's version of a paper published in *Transportation Research Part B*, 2016 — the version of record is available at doi.org/10.1016/j.trb.2016.04.009. It is typeset from the original sources in a uniform, publisher-neutral format; the content matches the published version.

Abstract

The integration of drones into civil airspace is one of the most challenging problems for the automation of the controlled airspace, and the optimization of the drone route is a key step for this process. In this paper, we optimize the route planning of a drone mission that consists of departing from an airport, flying over a set of mission way points and coming back to the initial airport. We assume that during the mission a set of piloted aircraft flies in the same airspace and thus the cost of the drone route depends on the air traffic and on the avoidance maneuvers used to prevent possible conflicts. Two Air Traffic Management techniques, i.e., routing and holding, are modeled in order to maintain a minimum separation between the drone and the piloted aircraft. The considered problem, called the Time Dependent Traveling Salesman Planning Problem in Controlled Airspace (TDTSPPCA), relates to the drone route planning phase and aims to minimize the total operational cost. Two heuristic algorithms are proposed for the solution of the problem. A mathematical formulation based on a particular version of the Time Dependent Traveling Salesman Problem, which allows holdings at mission way points, and a Branch and Cut algorithm are proposed for solving the TDTSPPCA to optimality. An additional formulation, based on a Travelling Salesman Problem variant that uses specific penalties to model the holding times, is proposed and a Cutting Plane algorithm is designed. Finally, computational experiments on real-world air traffic data from Milano Linate Terminal Maneuvering Area are reported to evaluate the performance of the proposed formulations and of the heuristic algorithms.

Keywords: Integer Programming, Time Dependent TSP, Drones, Air Traffic Management

1 Introduction

The *Time Dependent Traveling Salesman Planning Problem in Controlled Airspace* (TDTSPPCA) consists of finding the best order of visit of a drone over a set of mission way points (MWP), aiming at minimizing the operational cost and, at the same time, ensuring a minimum separation with respect to the planned traffic. Two air traffic management techniques are considered in order to impose such a minimum separation: the drone routing and the possibility of holding over a mission way point (also called target points). **These techniques are used in the planning phase as the one considered in this manuscript. The first one is a standard procedure, while the second one is often necessary to respect the minimum separation constraint or to reduce the operational cost (a holding of few minutes could reduce substantially the global travel time).**

The problem of finding the best order of visit of a set of points minimizing the total distance (cost, travel time) is called the Traveling Salesman Problem (TSP). It is one of the most investigated problems in transportation science and many applications and variants have been studied during the last decades. The temporal extension of the TSP, called *Time Dependent Traveling Salesman Problem* (TDTSP), has been addressed to deal with time dependent problems as traffic operations management in congested urban areas (we refer the interested reader to the seminal works of [1], [2], [3], [4] and [5]). The basic idea is that the cost of a path between two locations is time dependent: e.g., the travel time could significantly change during peak hours. The air traffic management in *controlled airspace* is intrinsically different from the traffic management in urban areas. On the ground, every single vehicle decides autonomously how to comply with the traffic rules. In the air, instead, a centralized manager (i.e., the ATC, Air Traffic Control) controls all the vehicles by using specific air traffic management techniques in order to comply with the air traffic rules. This holds in the controlled airspace, defined by the part of the airspace close to the airports, and in the en-route regions. Such rules, known as “Rules of the Air”, define a *minimum separation* to be applied between aircraft flying in the controlled airspace: the separation is a spacing (in space and and/or time) that must be applied in accordance to specific conditions (weather, type of traffic, etc). Thus, the main characteristic of the TDTSP arising in controlled airspace concerns the fact that paths between locations are forbidden during specific time periods due to the presence of other aircraft.

In order to apply these rules, the air traffic controllers have to solve complex and dynamic routing problems. Applications and software tools that solve routing problems from the pilot perspective are available both in literature and in commerce, e.g., GPS navigation, Traffic Collision Avoidance Systems and many others. Most of them uses state-based techniques in order to provide avoidance maneuvers or conflict free optimized trajectories (we refer the interested reader to [6], [7], [8] and [9] among other works). The methodology proposed in this manuscript tackles instead the routing problem from the ATC perspective. Our framework extends the Traveling Salesman Problem to the air transport system including the air traffic management techniques and it is based on a hybrid approach that merges Air Traffic Management (ATM) and Operations Research techniques. The solution of the TDTSPPCA allows an automated route calculation which includes the traffic constraints and reduces the workload of controllers and pilots.

As a case study, we adopt the so called “Intelligence, Surveillance, and Reconnaissance” (ISR) mission for drones (also called Unmanned Aircraft Systems). We choose this application since the drone operations are at a higher level of automation than the piloted aircraft. Potentially, the methodology proposed in this paper could be applied also to the piloted traffic as the level of automation in the traffic control increases. The use of the drones in military missions has proved their capability and versatility in different applications. There also are many civil applications that can be performed by the drones: for example, crop spraying or active volcano monitoring, crowd and traffic monitoring, search and rescue (SAR), fire monitoring and fire fighting. The use of drones in civil contexts will also produce several benefits by reducing risk for human crews, increasing mission efficiency and, not less important, reducing operations and personnel costs. The planning of an ISR mission in congested area is a complex problem for the air traffic controllers: several factors have to be considered as the level of priority of the traffic, the restriction of the airspace, the aircraft performances, etc. The aeronautical background and the modelling assumptions considered in this manuscript will be discussed in the next section.

1.1 Aeronautical background and modelling assumptions

The *Controlled Airspace* is the part of the airspace where the air traffic control service is provided, i.e., the airspace where the air traffic controllers ensure a minimum spatial and/or temporal separation between aircraft. To this aim, for each aircraft in their area of responsibility, the traffic controllers control: route, altitude, speed, rate of climb/descent, heading. A pilot plans his flight from an airport to another by requesting the air traffic control a route and a cruise level. The air traffic control approves the request or re-plans it. The *route* of an aircraft is defined by a sequence of navigation points and altitudes. An *airway* is a predefined succession of navigation points. An aircraft can follow an airway or can proceed directly between two navigation points belonging to different airways, if cleared by the air traffic control. The goal of the air traffic control is to ensure a safe and ordered flow of the air traffic in the controlled airspace. The controllers provide the legal minimum separation between aircraft using different techniques, the most used ones are: the vectoring, the speed control, the use of holdings, the rerouting, the use of different levels. The use of a particular technique depends on several factors as: the airspace layout, the aircraft performances, the human factors, the pilot training. The choices of the separation technique, of the routes and of the altitudes are strictly connected and affect the controlled air traffic and its evolution.

The controlled airspace is divided into “sectors”, typically each sector is managed by two controllers: the planner controller and the executive controller. The executive controller, who is in radio contact with the pilots, manages the air traffic in accordance with the planning defined by the planner controller. For each aircraft flying over a sector, the planner controller plans the routing, the altitude and other flight parameters considering several factors as flight plans, pilot requests, other traffic, weather and military restrictions. The planner controller coordinates the traffic between the sectors defining the route and the altitude (or flight level if above the transition altitude, e.g. 6000 ft). The planner controller accepts a traffic entry in his sector if the planned route and altitude are conflict free with respect to the other traffic. Typically, for each flight, this planning phase starts from 20 to 30 minutes before the estimated arrival time

into the sector and terminates when the flight leaves the sector.

When the planner controller receives the “inbound warning” (i.e., an alert regarding inbound traffic including the route, the flight level and the time estimate), an appropriate action on the incoming traffic is planned and applied if a conflict is identified. For example, a rerouting of the aircraft could be planned and executed by the executive controller (once the aircraft will entry the sector) or by the controllers of the preceding sectors. In congested sectors, the planner controller has to solve complex routing planning problems very often. In this paper, we focus on modeling this type of routing problems by designing effective algorithms able to solve them. We use a time dependent framework which includes routing and holdings in accordance to the traffic, and provides the planner controller a planned conflict free route.

Considering this complex scenario we make some basic assumptions. We investigate what is called in literature the *static case* of the problem, i.e., we assume that the air traffic that the drone may encounter is known in advance (in accordance to the planner controller time horizon). This assumption may reduce the realism of the problem solution, but it is a necessary step towards making headway in this line of research. This assumption is not over restrictive since we are optimizing the air traffic routing from a planner controller perspective, leaving to the executive controller and to the drone (auto)pilot the tactical management of the route.

We consider the drone as the traffic with the lowest priority. This is a realistic assumption since the drone missions are more flexible than the other aircraft. We assume that the conflict detection is performed from the air traffic control perspective and not from a cockpit perspective. This choice is particularly effective in the planning phase to model and use the air traffic controller techniques instead of the avoidance techniques as the TCAS (Traffic Collision Avoidance System) resolutions. The real air traffic is composed by the VFR (Visual Flight Rules) traffic, i.e., the aircraft that navigate by sight, and by the IFR (Instrumental Flight Rules) traffic, i.e., the aircraft that navigate using instruments. We consider only the IFR traffic, since the modeling of the VFR route is heavily subject to uncertainty.

The sequence of navigation points defining some aircraft route as SID (Standard Instrumental Departure) and STAR (Standard Arrival Route) must be flown in a defined order. We assume that in the drone route there are no restrictions on the visiting order of the mission way points. This represents the general case since the problem to be solved becomes easier by introducing restrictions in the visiting order.

Finally we assume that it is possible to perform a holding over each mission way point: typically the holdings are performed only in dedicated patterns, nevertheless the ISR missions require the possibility to hold over a target for operational reasons. We suppose that once a holding has been initiated by the drone, it will be able to complete such maneuver with no separation issues.

This is the ideal general context of a routing problem in controlled airspace. Moreover, there are many stochastic aspects that the air traffic controllers have to face during the traffic management, e.g. the real traffic may differ from the planned one, and the weather conditions may affect the routes and the avoidances. Since considering these aspects all together goes behind the scope of the paper, we put ourself in the context of planning a conflict free route considering the static case.

1.2 Paper contributions

We propose two new Mixed Integer Linear Programming (MILP) formulations of the Time Dependent Traveling Salesman Planning Problem in Controlled Airspace: the *Time-Based* model and the *Penalty-Based* model. Both formulations are capable of successfully dealing with real-world data, and specific algorithms are developed to speed up the solution time. Our methodology is based on a hybrid approach that shows that the problem of finding the best flying sequence to visit a set of mission way points in terms of total cost, and the problem of maintaining a minimum separation between the aircraft, can be combined using a temporal extension of the classical Traveling Salesman Problem (TSP). In other words, our goal is to solve the drone ATM planning phase by proposing an approach able to find optimized conflict-free routes. The output of the planning phase can then be the input for further tactical phases as the trajectory optimization or the real-time re-optimization of the routes (see Section 8 for further details on this issue).

The paper is organized as follows. In Section 2 we describe the literature on the related topics. In Section 3, we model the mission environment and the separation constraints, in addition we mathematically define the Time Dependent Traveling Salesman Planning Problem in Controlled Airspace (TDTSPPCA). In Section 4, we present two heuristic algorithms. In Section 5, we introduce the *Time-Based* ILP (Integer Linear Programming) model and propose a Branch and Cut algorithm for solving the TDTSPPCA to optimality. In Section 6, we present the *Penalty-Based* MILP (Mixed ILP) model and describe a Cutting Plane algorithm. Finally in Section 7, we report extensive computational experiments testing the proposed formulations and the heuristic algorithms on real air traffic data from Milano Linate (LIML) Terminal Maneuvering Area (TMA). In Section 8, we discuss possible extensions of the models in order to take into consideration additional real-world features of the problem. The conclusions are discussed in Section 9.

2 Related works

Given a list of points and the distances between each pair of them, the TSP consists of finding the shortest possible Hamiltonian circuit, i.e. the minimum distance route that visits each point exactly once. The TSP has been widely studied in the past years and many exact and heuristic algorithms have been developed to solve different routing problems. For a complete survey on the TSP see [10]. However, studies on the TSP with time-dependent travel costs (TDTSP) are fairly rare in the literature. Two main kinds of TDTSP have been proposed in the literature. The first one is characterized by arc costs that depend on the position of each arc in the Hamiltonian circuit, the second one by arc costs that depend on the time at which each arc is visited. As far as the first kind of TDTSP is concerned, we refer the interested reader to the seminal work [11].

The second kind of TDTSP arose to deal with traffic jams in congested urban areas: during peak periods the cost of a path between two points becomes higher than in normal conditions. We follow the second stream of research as this model is also useful to interpret air traffic using

a convenient time horizon compatible with aircraft operations. In fact, the cost of a drone route between two mission way points depends on the air traffic met by the drone, starting a path at a specific time. Considering a time interval of one minute and a mission duration of some hours, many exact algorithms proposed in the literature are not able to provide solutions in an acceptable computing time. [1] proposed a heuristic algorithm for the TDTSP, where two periods of the day present different travel costs, by adapting the classical savings algorithm developed by Clarke and Wright for the Vehicle Routing Problem. [12] proposed a MILP formulation of the TDTSP and of the related Time Dependent Vehicle Routing Problem TDVRP; the travel cost function is handled as a step function and several heuristic algorithms are presented to solve this problem.

In [13] a classification of the TDTSP is presented, considering old and new formulations obtained from a quadratic assignment model for the TDTSP. [14] used a dynamic programming heuristic algorithm to solve the TDTSP with a given starting time from the depot. [15] considered the bi-criteria TDVRP with time and area dependent travel speed. The minimization of the total vehicle operation time and the minimization of the total weighted tardiness were the conflicting objectives studied, and a MILP formulation for the problem was presented. This model was characterized by the waiting time at nodes that occurs when a vehicle awaits the next time interval for more rapid movement. A heuristic algorithm, called "bi-criteria-saving algorithm", is also presented to solve this problem. [16] described the derivation of travel time data from traffic information systems. The authors also tried to overcome the problem of the so-called non-passing property by using a smoothed travel time function for calculating the arrival time given a departure time. The non-passing property arises in the TDVRP when an earlier departure time of a vehicle must be coupled with an earlier arrival time, and vice versa. Computational results were reported on instances based on traffic information obtained from the city of Berlin. [2] solved the real-time TDVRP with time windows, using a heuristic algorithm that included methods for route construction; a technique to choose the optimal departure times was also developed. [17] considered a version of the TDTSP with time windows proposing an exact solution through a graph transformation. Such a problem is first transformed into an Asymmetric Generalized TSP and then into a Graphical Asymmetric TSP in order to apply exact known algorithms for the Mixed General Routing Problem. [18] proposed a branch and cut algorithm designed for a TDTSP used to model a production scheduling problem. In fact, since the set-up time between two jobs is a function of the completion time of the first job, the considered scheduling problem can be reformulated using a TDTSP approach. [18] introduced some families of valid inequalities used to strengthen the Linear Programming Relaxation of the proposed ILP formulation. [19] considered the TDVRP with time windows; the authors showed how to transform this problem into an Asymmetric Capacitated Vehicle Routing Problem in order to solve it with known exact or heuristic algorithms. [20] proposed a Tabu Search algorithm to solve the TDVRP and the related goods assignment problem: a real-world case of a warehousing company was used to illustrate the studied method. In [21], a study of the polytope associated with a particular TDTSP formulation is presented, and a branch-cut-and-price approach is applied to instances from the TSPLIB [22]. Finally, a routing problem for drones in controlled airspace has been presented in [23], where the authors propose

a Genetic Algorithm designed to manage a drone mission in civil airspace considering all the avoidance techniques previously mentioned.

3 Traffic separation modeling

The mission environment is modeled considering its key features: the airspace layout, the aircraft and their routes, the drone and the ATC avoidance techniques. The time is discretized, i.e. we consider the time as a succession of time steps separated by an interval chosen accordingly to the precision necessities. If t_{max} is an upper bound on the mission duration obtained by the drone endurance and Δt is the duration of a time step, the set $T = \{0, 1, 2, \dots, n_{ts} - 1\}$ is the set of time steps where $n_{ts} = t_{max}/\Delta t$ represents the number of time steps used.

We consider a set A of n aircraft flying during the period under investigation and an airspace defined by the set N_A of q navigation points used by the aircraft. Each navigation point $p \in N_A$ is characterized by a geographical position. Each aircraft $a \in A$ is characterized by a route $r_a \subseteq N_A$ defined by a succession of navigation points. The altitude and the time of the aircraft a passing through the navigation point p are called respectively l_p^a and t_p^a , moreover we define the set V_A as the set of all the pairs of navigation points and the set $R_A = \cup_{a \in A} r_a$ as the set of all aircraft routes. Finally we introduce the complete graph $G_A = \{N_A, V_A\}$ as the "Airspace Graph", i.e. the graph that defines the aircraft routes. This graph is directed and represents the airspace environment. Each aircraft, which flies in the modeled airspace during the mission, is moving over the Aircraft Graph G_A .

The surveillance mission consists of departing from an airport (mission way point 0), taking pictures over a set $M \setminus 0$ of $(m - 1)$ mission way points and coming back to the airport. Each mission way point $i \in M$ is characterized by a geographical position and an altitude. We assume that between each pair of mission way points a direct route is available. Let V_U be the set of all pairs of mission way points and we denote a specific pair with (i, j) . We introduce the complete graph $G_U = \{M, V_U\}$ as the "Drone Graph", i.e., the graph that defines the drone routes.

We suppose that a holding pattern is available over each mission way point: in other words the drone can overfly and, if necessary, can perform one or more orbits over a mission way point. We assume that the cost of a route can be approximated using the flight time, once the distances between the mission way points and the set P_{drone} of the drone mission parameters are known. These parameters depend on the drone performance and consist of the speed v_U (best range speed or best endurance speed, cruise speed), the rate of climb $\tilde{r}_c$ and the rate of descent $\tilde{r}_d$ that the drone would have used in the same routes in segregated airspace. Using such parameters, we define the flight time f_{ij} necessary to reach the mission way point j from the mission way point i . Due to the different performance during the climb and descent phases, we assume that the time from i to j is different from the time from j to i . Finally, since we assumed the drone as the airspace user with the lowest priority, if a conflict between an aircraft and the drone exists, the drone has to wait or to change its route and not vice versa (see [24] for further details on this topic). According to the airspace classification, a minimum separation must exist between each pair of aircraft (Drone included). Two ATC separation techniques are modeled and applied

to the drone planned route: the routing or the holding over a mission way point.

Let S be the set of all possible conflicts: a conflict $s \in S$ between the drone sub-route $(i, j) \in V_U$, and an aircraft route r_a occurs if at least one point exists on the drone sub-route such that its distance from r_a is less than 1000 ft on the vertical plane or 5 Nautical Miles (NM) on the horizontal plane. In this case, for the conflict s we define the Conflict Point p_c^s as the point closest to r_a . The time step in which the aircraft is overflying the closest point to p_c^s on its route is defined as the Conflict Time Step t_c^s . The avoidance of a conflict is modeled by forbidding the departure of the drone from the mission way point i to the mission way point j in the time steps involved in such conflict. For a given drone sub-route (i, j) , and aircraft route r_a , a time step is involved in a conflict s if a drone departure in such time step would determine an insufficient separation. Let us indicate with X_{ij}^s ($X_{ij}^s \subseteq T$) the conflict interval as the interval defined by the time steps involved in the conflict s over the sub-route (i, j) .

The identification of the time steps involved in a conflict for a given drone sub-route can be performed using simple geometric considerations: we use the conflict detection geometric algorithm presented in [23], where a complete analysis of all possible conflict cases is also reported. We report in Figure 1 the general case of conflict. The Conflict Zone consists of a sub-separation area defined by the drone and the aircraft route segments that are not separated from each other. Each conflict has an associated Conflict Zone: the Conflict Zone can not be occupied by the drone and an aircraft at the same time. Once checked that the vertical

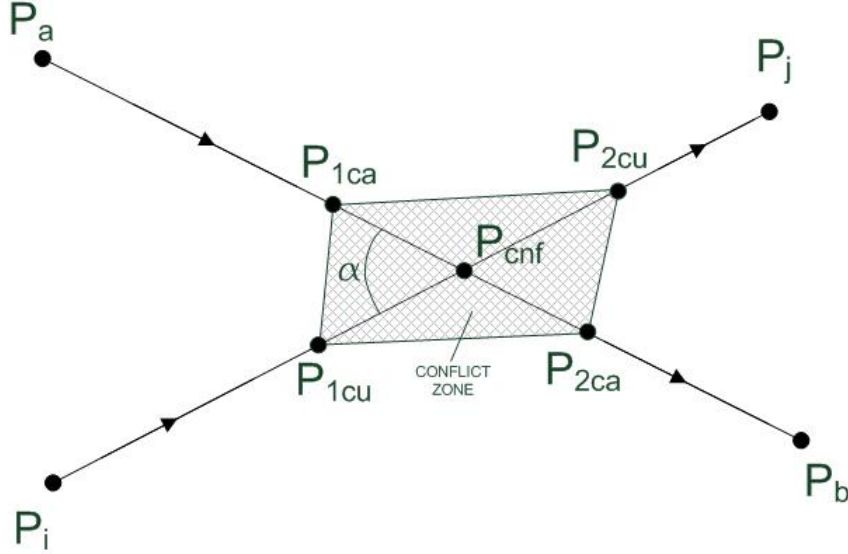


Figure 1: Conflict Zone

separation of 1000 ft does not exist between all points of the conflicting routes, their projection on the horizontal plain is used to identify the Conflict Zone. The segments (p_i, p_j) and (p_a, p_b) represent the projection on the plane of the drone sub-route (i, j) and of the aircraft route (a, b) , respectively. t_a and t_b are the time steps in which the aircraft is overflying p_a and p_b respectively. The point p_{1cu} is defined as the projection of the first point over the drone sub-route between p_i and p_c^s that presents a distance from the aircraft route of 1000 ft in the vertical direction or 5 NM in the horizontal direction, whichever happens first, moving from p_i to p_c^s . Similarly, the point p_{2cu} is the projection of the first point over the drone path between p_j and p_c that

presents a distance from the aircraft route of 1000 ft in the vertical direction or 5 NM in the horizontal direction, whichever happens first, moving from p_c^s to p_j . Analogously the points p_{1ca} and p_{2ca} are defined over the route of aircraft a ; the time steps in which aircraft a overflies such points are, respectively, t_{1ca} and t_{2ca} . The Conflict Zone is defined by the points p_{1cu} , p_{1ca} , p_{2cu} and p_{2ca} and it is comprised by two "sub-separation stretches": the first one is identified over the drone route between p_{1cu} and p_{2cu} and the second one over the aircraft route between p_{1ca} and p_{2ca} . If the drone is moving from p_i to p_j and the aircraft from p_a to p_b , the Conflict Interval over p_i starts after the last departing time step available before the conflict. That is, the departing time step from p_i such that the drone can arrive at p_{2cu} when the aircraft a is in the point p_{1ca} . The holding starts after the time step t'_{p_i} :

$$t'_{p_i} = \max\{0, t_a + \Delta t_{p_a, p_{1ca}} - \Delta t_{p_i, p_{2cu}}\} \quad (1)$$

Where $\Delta t_{p_i, p_{2cu}}$ is the time step interval necessary for the drone to fly from p_i to p_{2cu} maintaining P_{drone} , and $\Delta t_{p_a, p_{1ca}}$ is the time step interval necessary to the aircraft to fly from p_a to p_{1ca} . In the same way, the Conflict Interval over p_i finishes before the first re-departing time step available after the conflict. That is, the departing time step from p_i such that the drone can arrive at p_{1cu} when the aircraft a is in the point p_{2ca} . The holding finishes before the time step t''_{p_i} :

$$t''_{p_i} = \max\{0, t_a + \Delta t_{p_a, p_{2ca}} - \Delta t_{p_i, p_{1cu}}\} \quad (2)$$

Where $\Delta t_{p_a, p_{2ca}}$ is the time step interval necessary for the aircraft to fly from p_a to p_{2ca} , and $\Delta t_{p_i, p_{1cu}}$ is the time step interval necessary for the drone to fly from p_i to p_{1cu} . The dimension of the Conflict Zone depends on the angle α (that defines if the routes are diverging or converging) and on the altitudes of the drone and the aircraft on the mission way points. If $\alpha = 90^\circ$ and the altitudes on the points are the same, the Conflict Zone consists of a circle. In case $\alpha = 180^\circ$ (opposite routes) or $\alpha = 360^\circ$ (same routes) and the altitudes on the points are the same, the entire sub-route becomes a Conflict Zone during the aircraft passage. Using the notation reported above, it is possible to write the conflict interval X_{ij}^s defined by the time steps involved in the conflict s over the drone sub-route (i, j) :

$$X_{ij}^s =]t'_{p_i}, t''_{p_i}[\quad (3)$$

Considering all the conflicts over the drone sub-route (i, j) , we indicate with X_{ij} the set of the all time steps involved, i.e. the time steps in which a departure from the mission way point i to the mission way point j produces a conflict with an aircraft $a \in A$:

$$X_{ij} = \cup_{s \in S} X_{ij}^s \quad (4)$$

Considering that the air traffic and the related avoidance techniques used by the drone in a sub-route (i, j) depend on the time at which the drone starts to fly that sub-route, the problem combines two decisions: the choice of the best order of visiting a set of mission way points and the choice of the avoidance techniques that have to be used. These two dimensions can be

combined in a Time Dependent Asymmetric Travelling Salesman Problem where the cost of a path between two mission way points varies in time. Starting a sub-route at a specific time could produce a conflict, thus requiring an appropriate management, i.e., routing or holding.

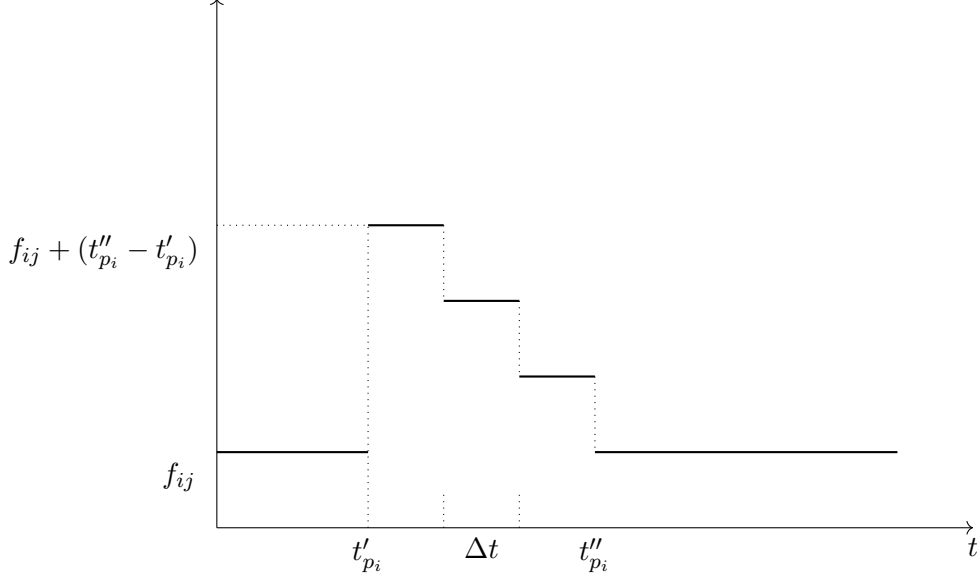


Figure 2: The time step function

Figure 2 describes the traversing time of the sub-route (i, j) as a function of the departing time step t : in case of no conflict ($t \notin X_{i,j}$) it corresponds to the flight time $f_{i,j}$ while, if a conflict occurs ($t \in X_{i,j}$), it corresponds to the flight time plus the holding time until the first time step without conflicts.

Using the previous notation, we can now formally define the *Time Dependent Traveling Salesman Planning Problem in Controlled Airspace* (TDTSPPCA) as follows: let $G = \{G_U, A, G_A, S, T\}$ be the mission conflict system, where at each mission way point $i \in M$, a holding pattern is permitted. Let β_i be the cost of a time step in the holding pattern associated with way point i , and γ the cost of a time step during the flight phase. Moreover, for a given set of flight parameters P_{drone} , let $f_{i,j}$ be the flight time from point i to point j , i.e., the flight time of arc (i, j) in the graph G_U , and k_{max} the maximum duration of a holding (with $k_{max} \geq 0$). For each pair (i, j) of way points, let $X_{i,j}$ be the set of time steps at which a departure from point i to point j produces a conflict with an aircraft $a \in A$. The goal consists of finding the route starting and coming back to the same prefixed mission way point (airport) and visiting all the other mission way points, i.e., a Hamiltonian circuit in the graph G_U . The problem asks for minimizing the mission cost, defined by the weighted sum of the global flying time and the global holding time, maintaining the minimum separation from the piloted traffic.

The TDTSPPCA is an asymmetric problem for two reasons: (i) the cost (flight time) of the sub-route (i, j) may be different from the cost of the sub-route (j, i) because the related mission way points may be at different altitudes; (ii) due to the presence of the air traffic, if the cost of the sub-routes is the same in both directions (symmetric case) two tours that present the

same sub-route but in opposite directions could have different total times. Another important characteristic of the problem follows from the properties of the considered objective function. If the holding time step costs $\beta_i, (i \in M)$ are all equal and the sequence of mission way points is fixed, the best way to visit them consists of waiting as less as possible over each of them. In other words, the route cost depends only on the sequence of the mission way points. This obviously does not hold when the holding costs are different.

4 Heuristic Algorithms

The nature of the considered problem is such that the air traffic situation changes at least every minute. This implies that for a 4 hours endurance drone mission with a time step of 1 minute, the number of time steps is not negligible. Moreover, since t_{max} is an upper bound defined using the drone endurance, it may be quite long with respect to the real mission duration. An appropriate tailoring of the set T becomes critical to solve effectively the TDTSPPCA, and thus heuristic approaches can be used to properly reduce the value of t_{max} , and hence the size of the set T . In addition the heuristic algorithms can be used to quickly find good feasible solutions for the TDTSPPCA (see the computational results of the manuscript).

Two heuristic algorithms have been designed to these aims: a Nearest Neighbour algorithm (NN) and an Asymmetric TSP (ATSP) route based algorithm (HTSP), plus a local search algorithm designed to further improve the solutions obtained.

The *ATSP route based algorithm* (HTSP) starts from the optimal TSP solution on the graph G_U , and the resulting tour is copied in the route vector. The holding vector is obtained by following the route defined by the route vector and performing the appropriate holdings: if a departing time step from a mission way point belongs to a Conflict Interval the holding corresponds to the wait until the first available departing time step (otherwise the holding is set to 0). In the *Nearest Neighbour algorithm* (NN), the solution vector is initially void and the algorithm starts initializing the route vector with the airport. The solution is then iteratively built, searching for the nearest mission way point in terms of flight time and holding cost (total cost). Once a mission way point is inserted into the route vector, the related holding to reach it is inserted in the holding vector. The algorithm stops when all the mission way points have been inserted in the route vector.

The neighbourhood used for the *Local Search algorithm* (LS), is the set of solutions obtained by performing, with respect to the current solution, all the exchanges between each pair of mission way points. We extended the classical TSP *2-opt procedure* which consists of trying to improve the solution by exchanging two of the solution arcs with other two not in the solution so as to rebuild a Hamiltonian circuit. With respect to the classical procedure, not only the cost of the exchange arcs has to be considered, but also the entire cost of the circuit must be recomputed. The Local Search algorithm is initialized through the route provided by one of the two previously mentioned heuristic algorithms. If a cheaper route can be found in the neighbourhood, the current solution is updated and the algorithm reiterates, otherwise the algorithm terminates with the current solution. Since the described algorithms require a very short computing time, a

good upper bound t_{ub} on the mission duration may be fastly computed as the smallest among the solutions values found by the considered algorithms. Using this upper bound, the number of time steps n_{ts} is possibly reduced. Detailed computational results of the heuristic algorithms are reported in Section 7.

5 Time-Based Model

In this section we present a time extension of a classical Asymmetric TSP (ATSP) formulation, a Variable Fixing Procedure to reduce the dimension of the model and a Branch and Cut Algorithm.

5.1 First Mathematical Formulation

Two types of binary variables are used to render the time dimension of the problem (variables x_{ij}^t) and to model the holding over the mission way points (variables y_i^t). Variable x_{ij}^t ($((i, j) \in V_U, t \in T)$) has value 1 if the drone starts to fly from mission way point i to mission way point j at time step t , and 0 otherwise. Variable y_i^t ($(i \in M, t \in T)$) takes value 1 if the drone arrives at the mission way point i at time step t , and 0 otherwise. By using these variables, for each mission way point $i \in M$, the arrival and departure times are given, respectively, by $\sum_{t \in T} ty_i^t$ and $\sum_{t \in T} \sum_{j \in M} tx_{ij}^t$. The Time-Based ILP model of the TDTSPPCA reads as follows:

$$\min \quad \gamma \sum_{t \in T} \sum_{(i,j) \in V_U} f_{ij} x_{ij}^t + \sum_{i \in M} \beta_i \left(\sum_{t \in T} \sum_{j \in M} t x_{ij}^t - \sum_{t \in T} t y_i^t \right) \quad (5)$$

$$\sum_{t \in T} \sum_{j \in M} x_{ji}^t = 1 \quad i \in M \quad (6)$$

$$\sum_{t \in T} \sum_{j \in M} x_{ij}^t = 1 \quad i \in M \quad (7)$$

$$\sum_{t \in T \setminus \{0\}} y_i^t = 1 \quad i \in M \setminus \{0\} \quad (8)$$

$$\sum_{j \in M, t - f_{ji} \geq 0} x_{ji}^{t - f_{ji}} \leq y_i^t \quad i \in M \setminus \{0\}, t \in T \quad (9)$$

$$\sum_{j \in M} x_{ij}^t \leq \sum_{t^*=0}^t y_i^{t^*} \quad i \in M, t \in T \quad (10)$$

$$\sum_{t \in T} \sum_{j \in M} t x_{ij}^t - \sum_{t \in T} t y_i^t \geq 0 \quad \forall i \in M \quad (11)$$

$$\sum_{t \in T} \sum_{j \in M} t x_{ij}^t - \sum_{t \in T} t y_i^t \leq k_{max} \quad \forall i \in M \quad (12)$$

$$x_{ij}^t = 0 \quad t : t + f_{ij} \geq t_{max}, (i, j) \in V_U \quad (13)$$

$$x_{ij}^t = 0 \quad t \in X_{i,j}, (i, j) \in V_U \quad (14)$$

$$y_0^0 = 1 \quad (15)$$

$$x_{ij}^t \in \{0, 1\} \quad (i, j) \in V_U, t \in T \quad (16)$$

$$y_i^t \in \{0, 1\} \quad i \in M, t \in T, \quad (17)$$

where γ and β_i ($i \in M$) are given coefficients (see Section 3.1). The objective function is composed by two terms. The first term represents the total flight time between the mission way points. The second term describes the holding duration over each mission way point as the difference between the departure and the arrival time steps. The coefficients γ and β_i make it possible to weigh the different phases of flight and holding: if rerouting is the preferred option with respect to holding, the coefficients β_i can be increased appropriately. If $\gamma = \beta_i = 1$ ($i \in M$) the objective function minimizes the total mission duration in terms of time steps and thus the impact of the ATC over the mission. To obtain the mission duration in time units it is necessary to consider the duration Δt of a generic time step. Constraints (6) and (7) are the “assignment” constraints, imposing that each mission way point i has one entering arc and one leaving arc, respectively. Constraints (8) ensure that each mission way point i has one arrival time. Constraints (9) and (10) joined with the initial condition (15), act as “Subtour Elimination Constraints” (forbidding possible partial circuits) and ensure the temporal coherence of the tour. In addition they force the departure from the airport at time step $t = 0$. Constraints (9) ensure that if the drone arrives at the mission way point i at time step t , it must have departed from the previous mission way point at an appropriate time step compatible with the flight time. Constraints (10) impose that if the drone leaves the mission way point i at time step

t , the arrival at i must occur at a previous or at the same time step. Both render the time coherence on the mission way points. These families of constraints make it possible to build the Hamiltonian circuit over the mission way points as in the ATSP. Constraints (14) impose no departure, according to the minimum separation constraint, in the forbidden time steps. Constraints (13) forbid departures after the last time step. Constraints (11) and (12) impose a lower bound (0) and an upper bound (κ_{max}) on the duration of the holding over a mission way point i , ensuring that the holding is consistent with the drone performances.

A Branch-and-Cut algorithm was developed based on the computation, at each node of the implicit enumeration tree, of the lower bound determined by the separation procedure described in Section 5.3. Before starting the algorithm, a preprocessing phase is performed. This phase consists of two steps. The first one is an upper bound computation on the mission duration (see Section 4) necessary to obtain a good value of t_{max} (small values of t_{max} reduce the number of variables of the model). The second step is a Variable Fixing procedure described in the following.

5.2 Variable Fixing procedure

The Variable Fixing procedure is based on the concept of “infeasible departure” time steps. An “Infeasible Departure” time step is a time step at which a departure of the drone cannot take place; the variable fixing procedure is composed by two reduction steps.

• Reduction Step 1

The first reduction step relies on the first time step at which it is possible to reach a given mission way point i . If t_i^{sp} is the first time step at which it is possible to arrive at point i , no departure from i to any other mission way point can take place before t_i^{sp} . The shortest path time from the airport (mission way point 0) to a mission way point i makes it possible to define its related t_i^{sp} . The Dijkstra algorithm ([25]) is an effective algorithm that is able to find the shortest path between the airport and each other mission way point. Note that in this context a shortest path algorithm must take into consideration the presence of air traffic. The cost of an arc in the Dijkstra algorithm has to consider two components: the holding until the first available time step and the flight time. The Dijkstra algorithm has been properly adapted to satisfy this constraint, by defining the cost of the arc (i, j) connecting point i with point j as the flight time (f_{ij}) from i to j plus the holding time at point i . The reduction step 1 can be expressed setting to 0 a set of variables as follow:

$$x_{ij}^t = 0 \quad i \in M, j \in M \setminus \{i\}, t < t_i^{sp}$$

• Reduction Step 2

The second reduction step imposes that no arrival at the airport can take place before the lower bound provided by the value t_{ATSP} of the optimal solution of the ATSP associated with the

graph G_U . This second reduction can be expressed setting to 0 a set of variables as follow:

$$x_{i0}^t = 0 \quad i \in M, t < t_{ATSP}$$

Both variable fixing steps substantially reduce the size of the model

5.3 The Branch and Cut Algorithm

In this section we describe the main features of the proposed Branch and Cut (BC) algorithm and of its *separation procedure*. Even though they are not necessary for the elimination of the sub-tours, the following inequalities can be added to formulation (5)-(17) in order to strengthen its Linear Programming (LP) relaxation (obtained by removing the integrality constraints on the values of the binary variables x_{ij}^t and y_i^t), and to determine better lower bounds for the TDTSPPCA:

$$\sum_{t \in T} \sum_{i \in Sub, j \in M \setminus Sub} x_{ij}^t \geq 1 \quad 2 \leq |Sub| \leq |M| - 2, Sub \subset M. \quad (18)$$

Such inequalities are a time extension of the well known Subtour Elimination Constraints (SEC) of the Dantzig-Fulkerson-Johnson model (see [26] for further details) for the Asymmetric TSP (ATSP) and they are potentially violated by the optimal LP solution. Since the number of inequalities (18) grows exponentially with the number $|M|$ of the mission way points, an effective technique to solve to optimality the LP relaxation of model (5)-(18), is to apply a separation procedure, i.e., to consider only a subset of such inequalities, and to insert iteratively additional inequalities which are violated by the solution of the current LP model. At each iteration, we find the optimal solution of the current LP model through the LP Solver Ilog Cplex 12.1, and then, to detect the inequalities (18) violated by this solution, we use the polynomial time separation algorithm proposed by Padberg and Rinaldi (see [27]) to separate the SEC for the Symmetric TSP (STSP). The Padberg and Rinaldi separation algorithm requires the computation of the maximum flow between each pair of vertices in a capacitated network defined by the original STSP complete undirected graph $G' = (M, E)$ and using as edge capacity the value of the corresponding variable in the optimal solution $\hat{x}_e$ ($e \in E$) of the LP relaxation of the Dantzig-Fulkerson-Johnson model of the STSP. If a maximum flow of value less than 1 is found, the corresponding violated SEC is added to the LP model.

A two step transformation is necessary to construct a solution $(\hat{x}_e, e \in E)$ of the LP relaxation of the Dantzig-Fulkerson-Johnson model of the STSP starting from the solution $\tilde{x}_{ij}^t$ ($((i, j) \in V_U, t \in T)$ of the LP relaxation of model (5)-(18) (initially only (5)-(17) and then, iteratively, some (18) inequalities). The first step constructs an ATSP solution $\bar{x}_{ij}$ ($((i, j) \in V_U)$ of the LP relaxation of the Dantzig-Fulkerson-Johnson model of the ATSP starting from $\tilde{x}_{ij}^t$ ($((i, j) \in V_U, t \in T)$):

$$\bar{x}_{ij} = \sum_{t \in T} \tilde{x}_{ij}^t \quad (i, j) \in V_U \quad (19)$$

The second step constructs a STSP solution $\hat{x}_e$ ($e \in E$) of the LP relaxation of the Dantzig-

Fulkerson-Johnson model of the STSP starting from $\bar{x}_{ij}$ $((i, j) \in V_U)$:

$$\hat{x}_e = \bar{x}_{ij} + \bar{x}_{ji} \quad e = (i, j) \in E \quad (20)$$

For further details on the second step, we address the interested reader to [28]. Whenever a violated SEC is found for the STSP, a corresponding violated inequality (18) can be easily obtained and added. This process is reiterated until no violated inequalities are detected, then the solution of the current LP model represents the optimal solution of the LP relaxation of model (5)-(18).

The inequalities (18) added at any node of the implicit enumeration tree are valid for all the other nodes because inequalities (18) are valid for the formulation (5)-(17). Thus, at a given node, all the inequalities so far generated are incorporated in the lower bound computation. This Branch-and-Cut algorithm has been implemented using the MILP Solver Ilog Cplex 12.1 with the default branching scheme. Even though this algorithm is able to solve to optimality many instances of the TDTSPPCA (see Section 7), the time dependence of the decision variables leads to weak lower bounds.

6 Penalty-Based Model

To avoid the weakness of the time dependent formulation, the TDTSPPCA is reformulated as a TSP with Penalties (TSPP) associated with the routes. These penalties represent an alternative way to model the holdings necessary to avoid the conflicts.

6.1 Second Mathematical Formulation

For a given mission way point $i \in M$, let $\tilde{H}_i$ be the set of partial routes that, starting from the airport, terminate at mission way point i and define a departure from i at a conflicting time step. Such conflict could be optimally avoided with a sequence of previous holdings: since the air traffic is known a priori, once a partial route is fixed, it is possible to find its optimal holdings distribution. Using this property, the new mathematical formulation extends the Dantzig-Fulkerson-Johnson ATSP model ([26]) by considering all possible conflicting partial routes and their related "penalty" constraints. Finally, denoting with H_i a given sequence of mission way points in $\tilde{H}_i$, we can define, for each point h in H_i , the corresponding optimal holding k_h . Note that, in this context, a sequence of mission way points conflicting in i is a part of a Hamiltonian circuit and that it includes all mission way points until the successor of i since the holding duration is related to an arc and not only to a mission way point.

Two types of decision variables are used. Binary variable x_{ij} $((i, j) \in V_U)$ takes value 1 if the drone uses the sub route (i, j) and 0 otherwise. Continuous variable p_i represents the holding over the mission way point i ($i \in M$). The MILP model of the TDTSPPCA reads as follows:

$$\min \quad \gamma \sum_{(i,j) \in V_U} f_{ij} x_{ij} + \sum_{i \in M} \beta_i p_i \quad (21)$$

$$\sum_{j \in M} x_{ji} = 1 \quad i \in M \quad (22)$$

$$\sum_{j \in M} x_{ij} = 1 \quad i \in M \quad (23)$$

$$\sum_{i \in Sub, j \in M \setminus Sub} x_{ij} \geq 1 \quad 2 \leq |Sub| \leq |M| - 2, Sub \subset M \quad (24)$$

$$\sum_{(lj) \in H_i} k_h(1 - x_{lj}) + p_h \geq k_h \quad h \in H_i, H_i \in \tilde{H}_i, i \in M \quad (25)$$

$$x_{ij} \in \{0, 1\} \quad (i, j) \in V_U \quad (26)$$

$$0 \leq p_i \leq k_{max} \quad i \in M \quad (27)$$

As in (5), the first term of the objective function represents the total flight time through the mission way points; the second term describes the global weighted holding over all the visited mission way points as a "penalty" that has to be added to the total cost of the route. To this end, it is necessary to add the "penalty" constraints (25) to the standard constraints (22)-(24) of the Dantzing-Fulkerson-Johnson ATSP formulation (see [26] for further details).

Constraints (25) relate the route and the penalties (holdings). They consider all possible partial routes H_i that could lead the drone to a conflict at point i . Clearly such constraints are exponential in number, but they can be separated using the procedure described in Section 6.2. To compute the coefficient k_h for each point h of a given sequence H_i , it is necessary to solve the associated *Holding Assignment Problem* (HAP). In the specific case of not weighted holdings, it is sufficient to wait only at the last mission way point before the conflict until the first available departure time step.

6.2 Holding Assignment Problem

Given a conflicting sequence H_i and the amount of holding time steps over its last reachable mission way point i , the goal of the HAP is to find the best way to assign such holding time steps over the mission way points until i . We define K as the set of the holding time steps that have to be assigned. By reassigning the holdings, the departure and arrival time steps of the following mission way points change. Thus, these time steps could correspond to conflicting time steps. As a consequence, it is necessary to consider the presence of further conflicts that can occur by changing the holdings. Nevertheless this solution could be better than that associated with smaller holdings over more sensitive mission way points. To model this kind of conflicts, further penalties are added in a manner similar to that used for the Penalty-Based model. For a given mission way point $i \in M$ and the corresponding conflicting sequence H_i , we define Q as the set of all possible assignments of the holding time steps of set K over the mission way points until i . A generic assignment $q \in Q$ can be represented by the appropriate pair (k, h) , with $k \in K$ and $h \in H_i$. Let k_h^q be the minimum holding duration over h for the assignment q , i.e., the number of holding time steps until the first available departure time step given the partial route H_i . We introduce a binary variable $z_{k,h}$ with value 1 if k holding time steps are assigned to the mission way point h and value 0 otherwise. Moreover we use the continuous variable p'_h to represent the further holding over h . Given $i \in M$, H_i , K and Q , the corresponding MILP

model of the HAP reads as follows:

$$\min \sum_{k \in K} \sum_{h \in H_i} \beta_h k z_{k,h} + \sum_{h \in H_i} \beta_h p'_h \quad (28)$$

$$\sum_{k \in K} \sum_{h \in H_i} k z_{k,h} + \sum_{h \in H_i} p'_h \geq |K| \quad (29)$$

$$\sum_{k \in K} k z_{k,h} + p'_h \leq k_{max} \quad h \in H_i \quad (30)$$

$$\sum_{k \in K} k z_{k,h} + p'_h \geq 0 \quad h \in H_i \quad (31)$$

$$\sum_{(k,h) \in q} B_1 k (1 - z_{k,h}) + p'_h \geq k_h^q \quad h \in H_i, q \in Q \quad (32)$$

$$z_{k,h} \in \{0, 1\} \quad k \in K, h \in H_i \quad (33)$$

$$u_h \in \{0, 1\} \quad h \in H_i \quad (34)$$

$$0 \leq p'_h \leq k_{max} \quad h \in H_i. \quad (35)$$

For the considered mission way point $i \in M$, the objective function (28) minimizes the total cost of the associated holdings by considering two components: the holdings due to the reassignment and the holdings corresponding to the presence of further conflicts. Constraints (29) ensure that the global holding duration is consistent with the number of time steps that have to be assigned. Constraints (30) and (31) represent the capacity constraints related, respectively, to the upper bound and the lower bound on the holding duration. Constraints (32) define the penalties by relating them with the reassignment. B_1 is a large number that can be set to k_h^q divided by k times the number of pairs (k, h) in q . To solve the HAP we separate the constraints (32) that are exponential in number. The algorithm starts by solving, through the MILP solver Ilog Cplex 12.1, the MILP model (28)-(35) without any constraint (32): if the solution does not imply further conflicts, the optimal solution of the HAP is found. Otherwise, at each mission way point $h \in H_i$ where a further conflict occurs, the associated number of holding time steps k until the first available time step is computed. Then the appropriate penalty k_h^q is added to the problem through the corresponding constraint (32) and the problem is solved again to optimality. The algorithm iterates until a non-conflicting solution is found. The optimal values of the variables $z_{k,h}$ ($k \in K, h \in H_i$) allow then to compute the set of coefficients k_h for each $h \in H_i$ as follows:

$$k_h = \sum_{k \in K} k z_{k,h}$$

6.3 The Cutting Plane Algorithm

In this section we describe the main features of the Cutting Plane (CP) algorithm proposed

for solving the TDTSPPCA. The algorithm starts by solving the MILP model defined by the first term of the objective function (21), by constraints (22)-(24) and constraints (26), obtained by the model (22)-(27) by omitting the contributions of the holdings variables p_i . This model corresponds to the "ATSP – relaxation" of the problem. Such relaxation is solved using the MILP Solver Ilog Cplex 12.1 and separating the SEC (24) through the Padberg and Rinaldi separation algorithm described in Section 5.3. If the ATSP route found presents no conflict, the optimal route for the TDTSPPCA is the ATSP route. Otherwise, a partial sequence H_i of mission way points can be defined as the partial ATSP route from the airport to the last mission way point $i \in M$ reachable without conflicts. The sets K and Q corresponding to the partial sequence H_i are defined and the related HAP is solved, using the procedure described in Section 6.2, in order to find the best distribution of the holdings k_h for the mission way points h belonging to the considered partial sequence H_i . The inequalities (25) corresponding to the mission way point i , the partial sequence H_i and the holdings k_h ($h \in H_i$) are added to the model and the problem is solved again to optimality. This procedure is iteratively repeated until a non-conflicting ATSP route is found.

7 Computational Results

We tested the algorithms proposed in the previous sections on real air traffic scenarios of the TMA (Terminal Maneuvering Area) of Milano Linate (ICAO code LIML), an important airport in the North of Italy with an average of 450 air traffic movements per day. We took into account the real air traffic of a day in the TMA of LIML using historical data regarding the aircraft routes. Chosen a traffic update interval of one minute, we have extrapolated the position and the altitude of each aircraft in the controlled airspace. In this way, we have been able to replicate such a traffic in order to simulate the drone mission to be planned. Computer Graphic tools have been used to model the air traffic scenario, to analyze the results and to rapidly evaluate the performances of the proposed algorithms. In this environment, navigation points such as Radio-Assistance and Fix Points (radial and distance by radio assistance) are reported; SID and STAR routes of the airport are modeled using graphic tools (see the appendix of the paper for further details on these graphic tools).

In order to generate the instances, we considered the air traffic data of LIML concerning 5 different scenarios. We used historical data representing the position and the altitude of departing, arriving and overflying aircraft during five different periods of time. The days considered were: the 11th of November 2009 from 05:30 until 13:30 UTC, the 22nd of August 2010 from 04:30 to 12:30 UTC, the 25th of August 2010 from 04:30 to 12:30 UTC, the 25th of August 2010 from 12:30 to 20:30 UTC and the 26th of August 2010 from 04:30 to 12:30 UTC. Considering these 5 different scenarios, we generated two types of instances: Medium-Short and Medium-Long instances. The first set consists of 15 instances considering for the 5 scenarios 10, 15 and 20 mission way points. The coordinates of the MWP were randomly selected among the navigation points located within a circle having center in LIML and ray of 20 NM. The second set consists of 15 instances considering for the 5 scenarios 30, 35 and 40 mission way points. The coordinates of these MWP are randomly selected among the navigation points located within a circle having

center in LIML and ray of 80 NM. To model a realistic update of the air traffic, a time step of one minute is set: this leads to a large number of time steps (more than 200) for a Medium-Long instance. The position and the altitude of each aircraft are updated at each time step.

We set the cost of the flight phase to 1 ($\gamma = 1$) and the cost of the ground holding to 2. Instead the cost of the airborne holdings is an integer number greater than 2 (and smaller than 10) that decreases with the distance from the airport: the basic idea is that a drone holding near the airport requires a more difficult management than a drone holding far from the airport, because the former zone is more sensitive since the traffic is more concentrated. In other words, we set the rerouting as the preferred option (using the smallest cost). The holdings follow, ordered by distance from the airport (far holdings are preferred). The only exception is the ground holding, which is a solution in between the rerouting and the airborne holdings. **We used these weights in order to penalize the delay at the beginning of the mission; this is particularly important in case of congested airports where the parking/taxi areas are limited resources.** In case minimizing the airborne holdings and of the flight phase represent instead an important goal, the weights of the ground base holding should be reduced and the weights of the airborne holdings and the flight phase should be increased. **In order to avoid (or limit) the holdings, very large weights can be used.**

As reference, we used a drone with performances similar to the General Atomic MK9 Reaper, called Predator B. This is a very successful drone that has proved its capabilities in different scenarios as combat, bombing, search and rescue and surveillance missions. Its primary users are the United States Air Force, the Royal Air Force and the Italian Air Force. Italian Reaper were used in Libya, in Kosovo, in Afghanistan and in “Mare Nostrum” (Mediterranean sea, migrants search and rescue operation). The length of the Predator B is 36 ft, its wingspan is 65 ft and its height is 12 ft. The Reaper has a fuel capacity of 4000 lb and an empty weight of 4901 lb. Its endurance is about 14 hours with a crew of 2 people in the ground station. Its maximum speed is 260 Kts (Knots) and the cruise speed is about 170 Kts. The service ceiling is 50,000 ft and its operational altitude is 25,000 ft. In our computational tests, we set the cruise speed to 180 Kts and the maximum climb/descent rate to 1500 ft/min. Moreover we set k_{max} to 20 minutes.

All the considered algorithms are run on a computer with an Intel Core Duo 2.31 GHz processor with 1.98 GB of RAM. In addition ILOG CPLEX 12.1 is used as LP and MILP solver. All the computing times reported in the following tables are expressed in seconds.

Tables 1 and 2 report the main characteristics of the instances and the results of the heuristic algorithms for the Medium-Short and the Medium-Long instances, respectively. The name of an instance is composed by: the ICAO code of the Airport (LIML), the date, the simulation beginning time plus "S" or "L" for Medium-Short or Medium-Long instances, respectively. The other characteristics are: the number of mission way points to visit ($|M|$), the number of simulated aircraft (n), the objective value of the ATSP solution (lower bound on the value of the optimal solution in controlled airspace) and the number of conflicts on the ATSP route. Then the tables report the heuristic solution values obtained by HTSP and by NN: for each of them, two objective values and the related percentage gaps are reported. The first objective

value (val) is the value of the solution found by the heuristic, the second one (LS) is the value found by the Local Search applied to such solution. The percentage gaps are computed between these upper bounds and the corresponding optimal solution value. For each group of instances the average gaps are also reported. The computing times of these algorithms are negligible (less than 1 second) and they are not reported in the tables. The tables show that there is no complete dominance between the two heuristic algorithms. As a consequence, in order to determine a good upper bound on the value of the optimal solution, we can apply, for each instance, both algorithms and keep the corresponding best (lowest) solution value. The last two columns (COMBO) report, for each instance, the objective value found by executing the heuristic algorithms in cascade and the related percentage gap. For the Medium-Short instances, the COMBO heuristic algorithm is able to find the optimal solution in 9 cases, but in some cases the gap can be quite big: the maximum gap is 68.75% for the last instances with 10 mission way points. For the Medium-Long instances, the COMBO heuristic is not able to find any optimal solution, but the average gap remains relatively small, around 10-15%. The first row of Table 1 can be interpreted as follows: the simulation starts the 11th of November at 05:30 considering a Medium-Short mission of 10 way points to visit. During the mission, 32 aircraft are simulated flying into the Linate Terminal Area. The optimal sequence of mission way points with no separation from the air traffic is represented by the ATSP solution. This solution has an objective function value of 29 minutes and generates two conflicts. Note that in the case of the ATSP solution, the objective function value corresponds to the flight time, this is not true in the case of a route with holdings, since the flight time and the holding are weighted. The objective function value increases to 43 by following the route defined by the ATSP solution and performing the necessary holdings to ensure the separation from the simulated air traffic (HTSP heuristic algorithm). Applying the Local Search procedure there is no improvement and the percentage gap is 38.71. A better solution can be obtained by using the NN heuristic algorithm with an the objective function value of 31 which is also the optimal solution value of the TDTTSPPCA.

Table 3 reports the results obtained by the exact Branch-and-Cut BC (see section 5.3) and the Cutting-Plane CP (see section 6.3) algorithms for the Medium-Short instances. The first two columns report the instance name, the number of mission way points to visit ($|M|$) and the optimal solution value found by both algorithms (“opt”). Then we report the results corresponding to the BC algorithm. In particular, we report two lower bounds (LB_1 and LB_2) with the corresponding computing times (time₁ and time₂). LB_1 is the value of the LP relaxation of model (5)-(17) while LB_2 is the value of the LP relaxation of the same model with the variable fixing procedure (see section 5.2) and the iterative addition of constraints (18) through the separation procedure (see section 5.3). Column (time) reports the computing time of the BC algorithm with a time limit of 1200 seconds (only for one instance the algorithm was not able to solve the instance to proven optimality). The following two columns report the number of nodes (nodes) and the number of inequalities (18) (cuts) added to model (5)-(17). Finally, for each instance, the table reports the solution time (time) and the number of iterations (it.) of the CP algorithm. For each group of instances, the average values are reported. Table 3 shows that, for this group of instances, the CP algorithm outperforms the BC algorithm, although

in two cases the BC algorithm is faster. The last row of the first group of instances of Table 3 can be interpreted as follows: the value of the optimal solution is 32, the BC algorithm is able to solve the problem in 104.8 seconds, and the CP algorithm in 71.2 seconds. With the variable fixing procedure and the addition of the inequalities (18), it is possible to strengthen the LP relaxation value of model (5)-(17) increasing the lower bound from 26 to 29. To optimally solve the problem, the BC algorithm explores 1290 nodes and 37 Cuts are generated. The CP algorithm performs 136 iterations to solve the considered instance.

Table 4 reports the results on the Medium-Long mission instances considering only the CP algorithm, since the BC algorithm is not effective for these instances. In columns 2 and 3 we report, for each instance, the number of mission way points and the optimal solution value. The following two columns report the computing time and the number of iterations needed to solve the problem to optimality. In order to obtain good feasible solutions within a shorter computing time, we executed the CP algorithm with a time limit equal to 1200 seconds. The corresponding results are reported in the columns "CP1200sec": for the instances not solved to optimality within this time limit, lb_2 reports the lower bound found by the algorithm within the time limit of 1200 seconds, and "gap" the corresponding percentage gap. It is important to note that the value of such gaps are already quite small: thus the CP algorithm is able to approximate the optimal solution in a reasonable computing time also in the case of Medium-Long instances. When the time limit of 1200 seconds is reached, the corresponding solution is not feasible: we can obtain a feasible solution by following the solution found until the first conflict is detected, and then applying the heuristic algorithms HTSP and NN and the local search procedure (LS). The last four columns of the table (CP 1200sec + Heur) report the values and the percentage gaps achieved by applying the heuristic algorithms (without and with the local search procedure LS) to the solution found within the time limit. In other words, in case of time limit, the infeasible solution found by the CP algorithm is completed using both algorithms HTSP and NN. The best value obtained among them is reported in column "val", and the corresponding solution is used as input for the LS procedure. The first row of the first group of instances reads as follows. The CP algorithm is able to optimally solve the problem of a mission of 30 way points in 6518.1 seconds with 1231 iterations. Using a time limit of 1200 seconds, the CP algorithm is able to approximate the optimal solution: the lower bound value found is 187 with a percentage gap of 1.58. Using the heuristic algorithms, the solution value is 214 (with a percentage gap of 12.63) and, by means of the LS procedure, it is possible to improve it to 213 (reducing the percentage gap to 12.11). In the second row, the CP algorithm is able to optimally solve the problem in less than 1200 seconds: for this reason, the last six columns are not filled.

Summarizing, the proposed heuristic algorithms are capable of finding good feasible solutions in short computing times but in same case the gap can be large. Optimal solutions can be obtained for the Medium-Short mission instances using both exact algorithm, i.e. the BC and the CP algorithms. The computing time increases but remains in all the cases (except 1 instance) below 1200 seconds. For the Medium-Long mission instances, higher computing times are necessary to solve the instances to proven optimality, the minimum and the maximum values are 45.1 and 123,327.4 seconds, respectively. Finally, since the computational effort in some cases can be quite high, we designed a hybrid heuristic approach (CP 1200sec + Heur) which is able to

obtain very good feasible solutions in less than 1200 seconds.

Instance					HTSP				NN				COMBO	
	$ M $	n	TSP	conf	val	gap	LS	gap	val	gap	LS	gap	val	gap
LIML 11/11 05:30 S	10	32	29	2	43	38.71	43	38.71	31	0.00	31	0.00	31	0.00
LIML 22/08 04:30 S	10	28	29	0	29	0.00	29	0.00	51	75.86	47	62.07	29	0.00
LIML 25/08 04:30 S	10	40	29	5	42	44.83	38	31.03	35	20.69	35	20.69	35	20.69
LIML 25/08 12:30 S	10	34	29	1	34	9.68	34	9.68	31	0.00	31	0.00	31	0.00
LIML 26/08 04:30 S	10	38	29	2	58	81.25	58	81.25	63	96.88	54	68.75	54	68.75
Averages						34.9		32.1		38.7		30.3		17.9
LIML 11/11 05:30 S	15	68	31	3	38	18.75	36	12.50	37	15.63	37	15.63	36	12.50
LIML 22/08 04:30 S	15	61	31	1	37	8.82	37	8.82	37	8.82	37	8.82	37	8.82
LIML 25/08 04:30 S	15	74	31	1	34	0.00	34	0.00	40	17.65	40	17.65	34	0.00
LIML 25/08 12:30 S	15	71	31	2	38	15.15	33	0.00	36	9.09	35	6.06	33	0.00
LIML 26/08 04:30 S	15	72	31	1	37	8.82	37	8.82	37	8.82	37	8.82	37	8.82
Averages						10.3		6.0		12.0		11.4		6.0
LIML 11/11 05:30 S	20	123	41	1	52	26.83	41	0.00	42	2.44	42	2.44	41	0.00
LIML 22/08 04:30 S	20	117	41	3	49	13.95	46	6.98	43	0.00	43	0.00	43	0.00
LIML 25/08 04:30 S	20	131	41	3	63	50.00	50	19.05	48	14.29	48	14.29	48	14.29
LIML 25/08 12:30 S	20	127	41	0	41	0.00	41	0.00	42	2.44	42	2.44	41	0.00
LIML 26/08 04:30 S	20	125	41	3	67	59.52	42	0.00	43	2.38	43	2.38	42	0.00
Averages						30.1		5.2		4.3		4.3		2.9

Table 1: Main characteristics and heuristic algorithm results for the Medium-Short Mission instances

Instance					HTSP				NN				COMBO	
	$ M $	n	TSP	conf	val	gap	LS	gap	val	gap	TS	gap	val	gap
LIML 11/11 05:30 L	30	213	179	5	214	12.63	214	12.63	229	20.53	229	20.53	214	12.63
LIML 22/08 04:30 L	30	207	179	4	207	9.52	207	9.52	233	23.28	233	23.28	207	9.52
LIML 25/08 04:30 L	30	220	179	3	203	9.73	203	9.73	215	16.22	211	14.05	203	9.73
LIML 25/08 12:30 L	30	214	179	5	221	13.92	221	13.92	216	11.34	212	9.28	212	9.28
LIML 26/08 04:30 L	30	216	179	4	205	8.47	205	8.47	211	11.64	211	11.64	205	8.47
Averages						10.9		10.9		16.6		15.8		9.9
LIML 11/11 05:30 L	35	229	191	6	225	14.80	218	11.22	232	18.37	231	17.86	218	11.22
LIML 22/08 04:30 L	35	226	191	4	236	21.65	236	21.65	253	30.41	253	30.41	236	21.65
LIML 25/08 04:30 L	35	271	191	4	216	9.09	216	9.09	244	23.23	231	16.67	216	9.09
LIML 25/08 12:30 L	35	245	191	7	253	30.41	220	13.40	253	30.41	253	30.41	220	13.40
LIML 26/08 04:30 L	35	250	191	2	228	14.57	225	13.07	244	22.61	244	22.61	225	13.07
Averages						18.1		13.7		25.0		23.6		13.7
LIML 11/11 05:30 L	40	247	199	9	228	9.09	228	9.09	259	23.92	258	23.44	228	9.09
LIML 22/08 04:30 L	40	232	199	6	262	27.18	251	21.84	280	35.92	280	35.92	251	21.84
LIML 25/08 04:30 L	40	282	199	6	220	6.80	220	6.80	263	27.67	263	27.67	220	6.80
LIML 25/08 12:30 L	40	257	199	10	226	10.78	224	9.80	263	28.92	253	24.02	224	9.80
LIML 26/08 04:30 L	40	261	199	4	232	12.62	230	11.65	250	21.36	247	19.90	230	11.65
Averages						13.3		11.8		27.6		26.2		11.8

Table 2: Main characteristics and heuristic algorithm results for the Medium-Long Mission instances

8 Possible extensions to the dynamic case

Future works will focus on the extension to the dynamic case, on the use of additional avoidance techniques and on the case of multi-drone missions. In the dynamic case, the air traffic situation is not known in advance. However, at a given step, the route cleared by the air traffic controllers is known for each aircraft. In this way, a reliable and realistic forecast of the air traffic for the next few minutes is available and can be used for the planning of the next mission way point to be visited by the drone. Moreover, the algorithms used for the static case, designed for solving the planning phase, can be easily extended to the dynamic case by defining, at each time step, two subsets of mission way points: the subset of points already visited and the subset

Instance			BC							CP	
	$ M $	opt	LB_1	time ₁	LB_2	time ₂	time	nodes	cuts	time	it.
LIML 11/11 05:30 S	10	31	26	0.0	29	0.0	3.8	38	6	6.7	34
LIML 22/08 04:30 S	10	29	26	0.0	29	0.1	1.5	0	6	0.1	1
LIML 25/08 04:30 S	10	29	26	0.0	29	0.0	0.9	0	7	0.9	8
LIML 25/08 12:30 S	10	31	26	0.0	29	0.0	5.2	30	6	19.5	53
LIML 26/08 04:30 S	10	32	26	0.0	29	0.1	104.8	1290	37	71.2	136
Averages							23.2	271.6	12.4	19.7	46.4
LIML 11/11 05:30 S	15	32	28	0.0	32	0.2	49.3	0	6	0.2	2
LIML 22/08 04:30 S	15	34	28	0.0	32	0.3	516.4	273	23	33.4	58
LIML 25/08 04:30 S	15	34	28	0.0	32	0.2	1200.0	1233	75	16.7	47
LIML 25/08 12:30 S	15	33	28	0.0	32	0.1	103.1	41	6	2.7	12
LIML 26/08 04:30 S	15	34	28	0.0	32	0.2	356.0	266	33	31.4	58
Averages							445.0	362.6	28.6	16.9	35.4
LIML 11/11 05:30 S	20	41	37	0.1	41	0.4	65.5	578	32	0.6	2
LIML 22/08 04:30 S	20	43	37	0.1	42	0.4	962.5	2864	44	69.7	118
LIML 25/08 04:30 S	20	42	36	0.1	42	0.3	32.7	259	23	6.3	15
LIML 25/08 12:30 S	20	41	37	0.1	41	0.3	368.3	5089	53	0.1	1
LIML 26/08 04:30 S	20	42	37	0.1	42	0.3	89.3	316	29	1.2	4
Averages							303.6	1821.2	36.2	15.6	28.0

Table 3: Comparison between algorithms BC and CP for the Medium-Short Mission instances.

Instance	$ M $	opt	CP		CP 1200sec		CP 1200sec + Heur			
			time	It.	lb_2	gap	val	gap	LS	gap
LIML 11/11 05:30 L	30	190	6518.1	1231	187	1.58	214	12.63	213	12.11
LIML 22/08 04:30 L	30	189	1176.2	632	*	*	*	*	*	*
LIML 25/08 04:30 L	30	185	45.1	81	*	*	*	*	*	*
LIML 25/08 12:30 L	30	194	2843.4	1987	188	3.09	221	13.92	221	13.92
LIML 26/08 04:30 L	30	189	1124.2	427	*	*	*	*	*	*
Averages			2341.4	871.6						
LIML 11/11 05:30 L	35	196	326.4	181	*	*	*	*	*	*
LIML 22/08 04:30 L	35	194	29.2	48	*	*	*	*	*	*
LIML 25/08 04:30 L	35	198	11546.3	771	194	2.02	218	10.10	215	8.59
LIML 25/08 12:30 L	35	194	11.1	22	*	*	*	*	*	*
LIML 26/08 04:30 L	35	199	4455.1	507	193	3.02	220	10.55	204	2.51
Averages			3273.6	305.8						
LIML 11/11 05:30 L	40	209	123327.4	2931	204	2.39	228	9.09	228	9.09
LIML 22/08 04:30 L	40	206	18408.3	949	203	1.46	262	27.18	242	17.48
LIML 25/08 04:30 L	40	206	17140.3	937	203	1.46	220	6.80	206	0.00
LIML 25/08 12:30 L	40	204	114.0	97	*	*	*	*	*	*
LIML 26/08 04:30 L	40	206	49235.2	867	201	2.43	232	12.62	232	12.62
Averages			41645.0	1156.2						

Table 4: Results of the CP algorithm for the Medium-Long Mission instances.

of points that have to be visited. At a given time step, once the decision variables associated with the mission way points of the first subset are set to the values corresponding to the part of the route already flown, the problem is solved and the next point among the points of the second subset is determined. This procedure is repeated until all the points have been visited. The main assumption of this approach is that the used algorithm must be able to solve the problem within a computing time smaller than the traffic updating interval. This is the case of the heuristic algorithms described in Section 4 and for the CP algorithm when applied to Medium-Short instances. Tactical avoidance techniques will be modeled and used in case of changes in the aircraft routes or in the controller clearances affecting the route until the next mission way point (this corresponds to the route in progress that can not be changed using the previously mentioned methodology). The use of the proposed approach based on the two subsets of points, will also allow to tactically change the point to be visited, to include new points or to modify the position of the old ones by using post-optimization procedures in order to define the best trajectory corresponding to the considered scenario. Rolling horizon techniques could be another approach to be investigated for the extension to the dynamic case. Thanks to the temporal discretization supported by the proposed approach, the algorithm can also be extended in order to consider moving targets (e.g., tracking of animals or ships) or targets dependent upon weather conditions. Finally the uncertainty can be captured in the arc costs, e.g., by giving higher costs to the arcs which are likely to produce conflicts with the air traffic.

9 Conclusions

In this paper we have studied the Time Dependent Traveling Salesman Planning Problem in Controlled Airspace. We have studied two different MILP models and designed two exact algorithms to solve them. The first one is a time dependent formulation, while the second one is based on a ATSP with penalties. **The proposed formulations and solution methods for the TDTSPPCA problem are the paper main contributions and represent the first step to effectively solve this complex and challenging real-world problem.** The proposed methodology is innovative since the conflict resolution uses an ATM approach, i.e., we model the avoidance through manoeuvres planned by the Air Traffic Controllers in order to optimize a conflict free route. Heuristic algorithms have also been proposed and tested. Computational results have shown that the proposed exact and heuristic algorithms are able to tackle real-world instances in short computing times. **Since this is the first manuscript dealing with the Time Dependent Traveling Salesman Planning Problem in Controlled Airspace, many assumptions have been necessarily made. Future research will be in the direction of considering less restrictive assumptions, enriching in such a way the models and the solution algorithms.**

Acknowledgment

The authors thank three anonymous referees for their useful comments, which helped in improving the presentation and the results of the manuscript.

References

References

- [1] J.E. Beasley. Adapting the savings algorithm for varying inter-customer travel times. *Omega*, 9(6): 658 – 659, 1981.
- [2] H. Chen, C. Hsueh, and M. Chang. The real-time time-dependent vehicle routing problem. *Transportation Research Part E: Logistics and Transportation Review*, 42(5):383–408, 2006.
- [3] I. Sánchez-Díaz, J. Holguín-Veras, and X. Ban. A time-dependent freight tour synthesis model. *Transportation Research Part B: Methodological*, 78:144 – 168, 2015.
- [4] W. Lam, Z. Li, H. Huang, and S. Wong. Modeling time-dependent travel choice problems in road networks with multiple user classes and multiple parking facilities. *Transportation Research Part B: Methodological*, 40(5):368 – 395, 2006.
- [5] A. Franceschetti, D. Honhon, T. Van Woensel, T. Bektaş, and G. Laporte. The time-dependent pollution-routing problem. *Transportation Research Part B: Methodological*, 56:265 – 293, 2013.
- [6] J. Le Ny, E. Feron, and E. Frazzoli. On the dubins traveling salesman problem. *Automatic Control, IEEE Transactions on*, 57(1):265–270, 2012.
- [7] J.T. Isaacs, D.J. Klein, and J.P. Hespanha. Algorithms for the traveling salesman problem with neighborhoods involving a dubins vehicle. In *American Control Conference (ACC), 2011*, pages 1704–1709, 2011.
- [8] K. Sundar and S. Rathinam. Algorithms for routing an unmanned aerial vehicle in the presence of refueling depots. *Automation Science and Engineering, IEEE Transactions on*, 11(1):287–294, 2014.
- [9] E. Sonmezocak and S. Kurt. Optimum route planning and scheduling for unmanned aerial vehicles. *Naval Postgraduate School, Monterey, California*, 2008.
- [10] G. Gutin and Punnen (EDS.). The Traveling Salesman Problem and Its Variations. *Springer*, 2002.
- [11] J. C. Picard and M. Queyranne. The Time-Dependent Traveling Salesman Problem and Its Application to the Tardiness Problem in One-Machine Scheduling. *Operations Research*, 26:86–110, 1978.
- [12] C. Malandraki and M. S. Daskin. Time Dependent Vehicle Routing Problems: Formulations, Properties and Heuristic Algorithms. *Transportation Science*, 26(3):185–200, 1992.
- [13] L. Gouveia and S. Voss. A classification of formulations for the (time-dependent) traveling salesman problem. *European Journal of Operational Research*, 83(1):69 – 82, 1995.
- [14] C. Malandraki and R.B. Dial. A restricted dynamic programming heuristic algorithm for the time dependent traveling salesman problem. *European Journal of Operational Research*, 90(1):45 – 55, 1996.
- [15] Y. Park. A solution of the bicriteria vehicle scheduling problems with time and area-dependent travel speeds. *Computers & Industrial Engineering*, 38(1):173–187, 2000.
- [16] B. Fleischmann, M. Gietz, and S. Gnutzmann. Time-Varying Travel Times in Vehicle Routing. *Transportation Science*, 38(2):160–173, 2004.

- [17] J. Albiach, J. Sanchis, and D. Soler. An asymmetric TSP with time windows and with time-dependent travel times and costs: An exact solution through a graph transformation. *European Journal of Operational Research*, 189(3):789–802, 2008.
- [18] G. Stecco, J. Cordeau, and E. Moretti. A branch-and-cut algorithm for a production scheduling problem with sequence-dependent and time-dependent setup times. *Computers & Operations Research*, 35(8):2635–2655, 2007.
- [19] D. Soler, J. Albiach, and E. Martinez. A way to optimally solve a time-dependent Vehicle Routing Problem with Time Windows. *Operations Research Letters*, 37(1):37–42, 2009.
- [20] Y. Kuo, C. Wang, and P. Chuang. Optimizing goods assignment and the vehicle routing problem with time-dependent travel speeds. *Computers & Industrial Engineering*, 57(4):1385–1392, 2009.
- [21] H. Abeledo, R. Fukasawa, A. Pessoa, and E. Uchoa. The time dependent traveling salesman problem: polyhedra and algorithm. *Mathematical Programming Computation*, 5(1):27–55, 2013.
- [22] G. Reinelt. TSPLIB-A traveling salesman problem library. *ORSA Journal on Computing*, 3(4):376–384, 1991.
- [23] C. A. Persiani and S. Bagassi. Route planner for unmanned aerial system insertion in civil non-segregated airspace. *Journal of Aerospace Engineering (to appear)*, 2013.
- [24] E. Mueller, C. Santiago, A. Cone, and T. Lauderdale. Effects of UAS Performance Characteristics, Altitude, and Mitigation Concepts on Aircraft Encounters and Delays. *Air Traffic Control Quarterly*, 21(1), 2013.
- [25] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, 1959.
- [26] G. Dantzig, R. Fulkerson, and S. Johnson. Solution of a large-scale traveling-salesman problem. *Operations Research*, 2(4):pp. 393–410, 1954.
- [27] M. Padberg and G. Rinaldi. A Branch-and-Cut Algorithm for the Resolution of Large-Scale Symmetric Traveling Salesman Problems. *SIAM Review*, 33(1):60, 1991.
- [28] M. Fischetti and P. Toth. A polyhedral approach to the asymmetric traveling salesman problem. *Management Science*, 43(11):1520–1536, 1997.

A Example of avoidance using routing and holding maneuvers

We report an example of avoidance solved using the methodologies proposed in the manuscript. The Medium-Long instance of the 11th of November 2009 is reported in Figures A.3(a) and A.3(b). In Figure A.3(a) the TSP route is shown. The drone is represented by a black triangle while the aircraft are represented by a white triangle. Dashed segments represent the aircraft routes and continuous segments represent the drone route. In this case the time required for the drone to overfly all 20 mission way points is 162 minutes (TSP optimal solution value) and 4 conflicts between the drone and the aircraft are detected (represented by black bullets, one of them is covered by the flight SMX5294). In Figure A.3(b) the route without conflicts is reported. The duration of the mission in the controlled airspace is 171 minutes: a rerouting

makes it possible to avoid the piloted air traffic.

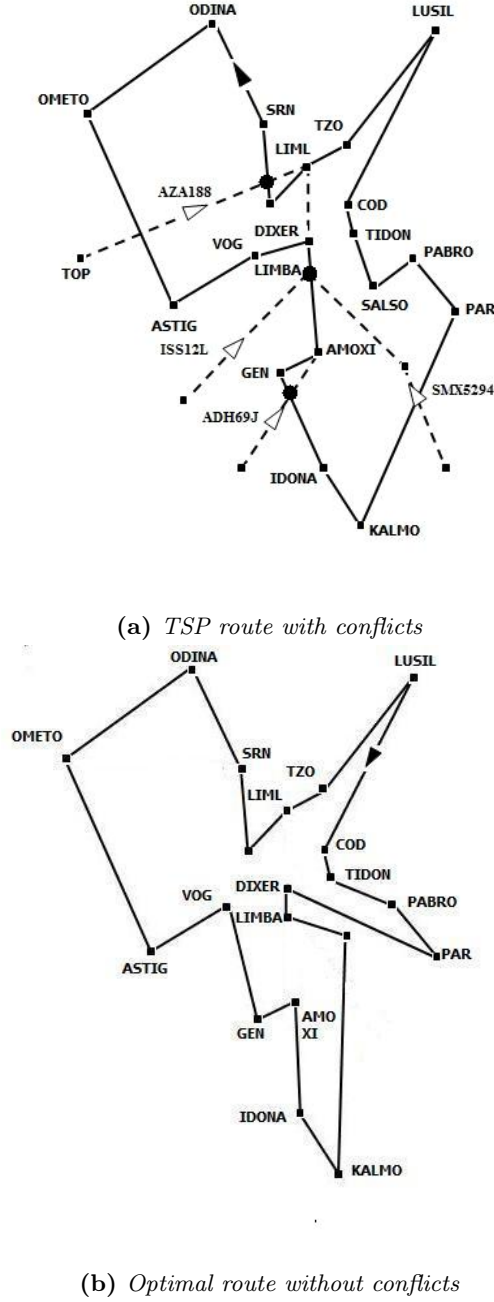


Figure 3: Example of routes

An interesting example of drone conflict resolution in such mission is reported in Figure 4. The points AMOXI, LIMBA and DIXER are lined up to Runway 36 of Linate airport; the arriving aircraft coming from South follow this route. Looking at the figures in sequence it is possible to recognize an arrival sequence concerning flights AZA2036, ACL324 and AZA2032. The drone route includes the path between DIXER and LIMBA in the opposite direction. This path is performed by the drone between ACL324 and AZA2032 without separation minima infringement.

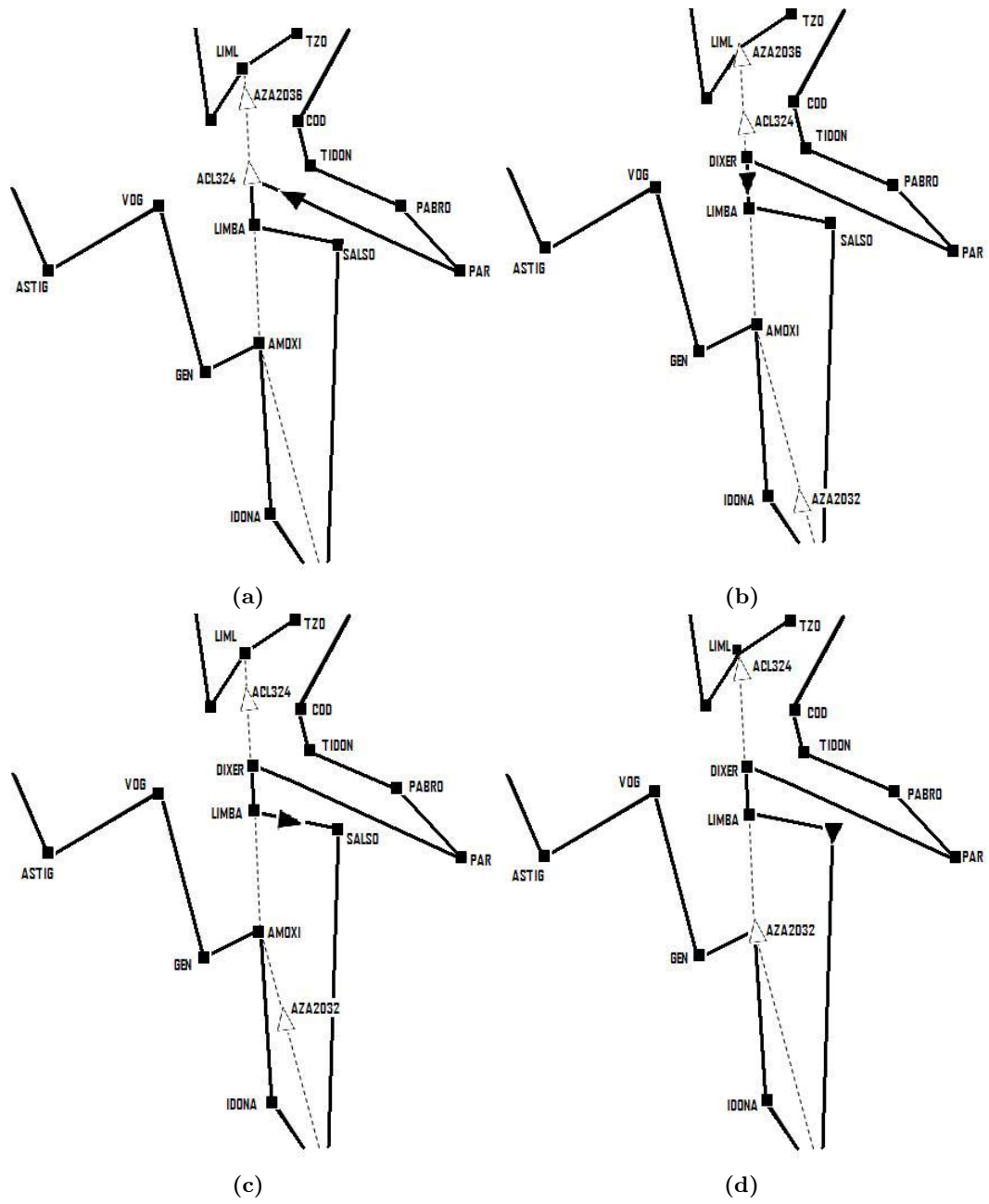


Figure 4: Example of Avoidance